RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name : CATHERINE NIVEDITA V

Year / Branch / Section : 2ND YEAR / INFORMATION TECHNOLOGY / IT A

University Register No. : 2116251001037

College Roll No. : 251001037

Semester : THIRD SEMESTER

Academic Year : 2026-2027

INDEX PAGE

Exp. No.	Name of the Experiment	Date of Completion
1	Finding Complexity using Counter Method	06-08-2026
2	Finding Complexity using Counter Method	06-08-2026
3	Finding Complexity using Counter Method	06-08-2026
4	Finding Complexity using Counter Method	06-08-2026
5	Finding Complexity using Counter Method	06-08-2026
6	Coin Change (Greedy)	27-08-2026
7	Cookies Problem (Greedy)	27-08-2026
8	Burger Problem (Greedy)	30-08-2026
9	Array Sum Maximum	27-08-2026
10	Product of Array Elements - Minimum	27-08-2026
11	Number of Zeros in a Given Array	06-08-2026
12	Majority Element	06-08-2026
13	Finding Floor Value	06-08-2026
14	Two Elements Sum to X	06-08-2026
15	Implementation of Quick Sort	30-09-2026
16	Playing with Numbers	17-09-2026
17	Playing with Chessboard	17-09-2026
18	Longest Common Subsequence	17-09-2026
19	Longest Non-decreasing Subsequence	17-09-2026
20	Finding Duplicates - $O(n^2)$	17-09-2026
21	Finding Duplicates - O(n)	17-09-2026
22	Intersection of 2 Sorted Arrays - O(m*n)	17-09-2026
23	Intersection of 2 Sorted Arrays - O(m+n)	17-09-2026
24	Pair with Difference - $O(n^2)$	17-09-2026
25	Pair with Difference - O(n)	17-09-2026

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 10:28 AM
State	Finished
Completed on	Thursday, 6 August 2026, 10:35 AM
Time taken	6 mins 39 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer: (penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int counter = 0;
    int i = 1;
    counter++;          // i = 1

    int s = 1;
    counter++;          // s = 1

    while (1) {
        counter++;      // while condition: s <= n
        if (!(s <= n))
            break;
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 10:35 AM
State	Finished
Completed on	Thursday, 6 August 2026, 10:43 AM
Time taken	7 mins 34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
void func(int n){
    int c=0;
    c++;
    if(n==1){
        c++;
    }
    else{
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                c++;
                c++;
                c++;
                c++;
                break;
            }
            c++;
        }
        c++;
    }
    printf("%d",c);
}
int main(){
    int n;
    scanf("%d",&n);
    func(n);
}
```

	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 10:43 AM
State	Finished
Completed on	Thursday, 6 August 2026, 10:48 AM
Time taken	5 mins 6 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include<stdio.h>
void Factor(int num){
    int c=0;
    for(int i=1;i<=num;++i){
        c++;
        c++;
        if(num%i==0){
            c++;
        }
    }
    c++;
    printf("%d",c);
}
int main(){
    int n;
    scanf("%d",&n);
    Factor(n);
}
```

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 10:49 AM
State	Finished
Completed on	Thursday, 6 August 2026, 10:55 AM
Time taken	6 mins 48 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include<stdio.h>
void function(int n){
    int counter=0;
    int c=0;
    counter++;
    for(int i=n/2;i<n;i++){
        counter++;
        for(int j=1;j<n;j=2*j){
            counter++;
            for(int k=1;k<n;k=k*2){
                counter++;
                c++;
                counter++;
            }
            counter++;
        }
            counter++;
        }
        counter++;
        printf("%d",counter);
}
int main(){
    int n;
    scanf("%d",&n);
    function(n);
}
```

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 10:57 AM
State	Finished
Completed on	Thursday, 6 August 2026, 11:13 AM
Time taken	15 mins 52 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include<stdio.h>
void reverse(int n){
    int c=0;
    int rev=0,remainder;
    c++;
    while(n!=0){
        c++;
        remainder=n%10;
        c++;
        rev=rev*10+remainder;
        c++;
        n/=10;
        c++;
    }
    c++;
    c++;
    printf("%d",c);
}
int main(){
    int n;
    scanf("%d",&n);
    reverse(n);
}
```

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 27 August 2026, 10:08 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:09 AM
Time taken	1 min 7 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int V;
    scanf("%d", &V);

    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;
    int i;

    for (i = 0; i < 9; i++) {
        count += V / denominations[i];
        V = V % denominations[i];
    }

    printf("%d\n", count);

    return 0;
}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 27 August 2026, 10:09 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:10 AM
Time taken	52 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a > *(int *)b) - (*(int *)a < *(int *)b);
}

int main() {
    int n, m;

    // Number of children
    scanf("%d", &n);

    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }

    // Number of cookies
    scanf("%d", &m);

    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }

    // Sort greed factors and cookie sizes
    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int child = 0;
    int cookie = 0;

    // Greedy matching
    while (child < n && cookie < m) {
        if (s[cookie] >= g[child]) {
            child++;
        }
```

```
39          cookie++;
40      }
41
42      printf("%d\n", child);
43
44      return 0;
45  }
46
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 27 August 2026, 10:11 AM
State	Finished
Completed on	Sunday, 30 August 2026, 7:40 PM
Time taken	3 days 9 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3

burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance

he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

3

5 10 7

Sample Output

76

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
#include<math.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s=0;
    for(int i=0;i<n;i++){
        s+=(pow(n,i))*arr[i];
    }
    printf("%d",s);
}
```

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 27 August 2026, 10:17 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:18 AM
Time taken	55 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;

    return (x > y) - (x < y);
}

int main() {
    int n;
    scanf("%d", &n);

    long long arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    // Sort in ascending order
    qsort(arr, n, sizeof(long long), compare);

    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += arr[i] * i;
    }

    printf("%lld\n", sum);

    return 0;
}
```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔

	Input	Expected	Got	
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 27 August 2026, 10:18 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:29 AM
Time taken	10 mins 35 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int ascending(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;
    return (x > y) - (x < y);
}

int descending(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;
    return (y > x) - (y < x);
}

int main() {
    int n;
    scanf("%d", &n);

    long long array_One[n];
    long long array_Two[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_One[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_Two[i]);
    }

    // Sort first array in ascending order
    qsort(array_One, n, sizeof(long long), ascending);

    // Sort second array in descending order
    qsort(array_Two, n, sizeof(long long), descending);

    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += array_One[i] * array_Two[i];
    }

    printf("%lld\n", sum);

    return 0;
}

```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 11:25 AM
State	Finished
Completed on	Thursday, 6 August 2026, 11:26 AM
Time taken	40 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Divide and Conquer function to find the index of the first occurrence of 0
int countZeros(int arr[], int low, int high, int m) {
    if (high >= low) {
        int mid = low + (high - low) / 2;

        // Check if mid is the first occurrence of 0
        // It is the first 0 if it's at index 0 or the element before it is 1
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
            return m - mid; // Total size - starting index of 0 gives count of 0s
        }

        // If element is 1, the first 0 must be on the right side
        if (arr[mid] == 1) {
            return countZeros(arr, mid + 1, high, m);
        }
        // Otherwise, the first 0 is on the left side
        else {
            return countZeros(arr, low, mid - 1, m);
        }
    }

    // If no 0 is found (all elements are 1)
    return 0;
}

int main() {
    int m;
    if (scanf("%d", &m) != 1) return 0;

    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    // Special case: if the first element is 1 and last is 1, there are 0 zeros.
    // If the first element is 0, all elements are zeros.
    if (arr[0] == 0) {
        printf("%d\n", m);
    } else if (arr[m - 1] == 1) {
        printf("0\n");
    } else {
        int zeros = countZeros(arr, 0, m - 1, m);
        printf("%d\n", zeros);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 6 August 2026, 11:26 AM
State	Finished
Completed on	Thursday, 6 August 2026, 11:26 AM
Time taken	27 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Function using Boyer-Moore Voting Algorithm to find the majority element
int majorityElement(int nums[], int n) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int result = majorityElement(nums, n);
    printf("%d\n", result);

    return 0;
```

```
35 }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT C2

Started on	Thursday, 6 August 2026, 11:26 AM
State	Finished
Completed on	Thursday, 6 August 2026, 11:27 AM
Time taken	28 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Divide and Conquer / Binary Search approach to find the floor of x
int findFloor(int arr[], int low, int high, int x) {
    // If low exceeds high, no floor exists (return -1 or handle appropriately)
    if (low > high) {
        return -1;
    }

    // If x is greater than or equal to the last element, then the last element is the floor
    if (x >= arr[high]) {
        return arr[high];
    }

    // If x is smaller than the first element, no floor exists in this range
    if (x < arr[low]) {
        return -1;
    }

    int mid = low + (high - low) / 2;

    // If element is exact match, it is the floor
    if (arr[mid] == x) {
        return arr[mid];
    }

    // If mid element is smaller than x, then mid could be a potential floor,
    // but we should check if there's a larger element on the right side.
    if (arr[mid] < x) {
        int floorVal = findFloor(arr, mid + 1, high, x);
        return (floorVal != -1) ? floorVal : arr[mid];
    }

    // If mid element is greater than x, the floor must lie in the left half
    return findFloor(arr, low, mid - 1, x);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int floor = findFloor(arr, 0, n - 1, x);
    if (floor != -1) {
        printf("%d\n", floor);
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT C2

Started on	Thursday, 6 August 2026, 11:24 AM
State	Finished
Completed on	Thursday, 6 August 2026, 11:25 AM
Time taken	59 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdbool.h>

// Recursive function using divide and conquer (binary search approach)
// to find if an element 'target' exists in arr[low...high]
bool binarySearch(int arr[], int low, int high, int target, int *foundVal) {
    if (low > high)
        return false;

    int mid = low + (high - low) / 2;
    if (arr[mid] == target) {
        *foundVal = arr[mid];
        return true;
    }
    else if (arr[mid] > target) {
        return binarySearch(arr, low, mid - 1, target, foundVal);
    }
    else {
        return binarySearch(arr, mid + 1, high, target, foundVal);
    }
}

// Recursive divide and conquer function to check pairs
bool findPair(int arr[], int low, int high, int x, int *num1, int *num2) {
    if (low >= high)
        return false;

    // Divide step: pick the middle element and search for (x - arr[mid]) in the rest
    int mid = low + (high - low) / 2;
    int complement = x - arr[mid];
    int foundVal = -1;

    // Search for complement using binary search (excluding mid itself)
    // We can search either in [low, mid-1] or [mid+1, high] depending on complement value
    if (complement < arr[mid]) {
        if (binarySearch(arr, low, mid - 1, complement, &foundVal)) {
            *num1 = foundVal;
            *num2 = arr[mid];
            return true;
        }
    } else {
        if (binarySearch(arr, mid + 1, high, complement, &foundVal)) {
            *num1 = arr[mid];
            *num2 = foundVal;
            return true;
        }
    }

    // Conquer step: recursively check left and right halves
    if (findPair(arr, low, mid - 1, x, num1, num2))
        return true;

```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 13 August 2026, 10:23 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 3:15 PM
Time taken	48 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int i = low, j = high;
        int pivot = a[low];
        int temp;

        while (i < j) {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main() {
    int n, i;
    int a[100];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:17 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:23 AM
Time taken	5 mins 52 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format
First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

long long countWays(int n) {
    if (n < 0) return 0;

    // dp[i] will store the number of ways to represent integer i
    long long dp[n + 1];

    // Base cases
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] = 0;
        // Add ways from (i - 1) if valid
        if (i >= 1) {
            dp[i] += dp[i - 1];
        }
        // Add ways from (i - 3) if valid
        if (i >= 3) {
            dp[i] += dp[i - 3];
        }
    }

    return dp[n];
}

int main() {
    int n;
    // Reading input format as specified
    if (scanf("%d", &n) == 1) {
        printf("%lld\n", countWays(n));
    }
    return 0;
```

```
34 }
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT C2

Started on	Thursday, 17 September 2026, 10:23 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:25 AM
Time taken	2 mins 30 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    // Dynamically allocate memory for the grid and dp tables
    int **grid = (int **)malloc(n * sizeof(int *));
    int **dp = (int **)malloc(n * sizeof(int *));

    for (int i = 0; i < n; i++) {
        grid[i] = (int *)malloc(n * sizeof(int));
        dp[i] = (int *)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            scanf("%d", &grid[i][j]);
        }
    }

    // Base case for the starting position
    dp[0][0] = grid[0][0];

    // Initialize the first row (can only move right)
    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }

    // Initialize the first column (can only move down)
    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }

    // Fill the rest of the DP table using the recurrence relation
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            int max_prev = (dp[i - 1][j] > dp[i][j - 1]) ? dp[i - 1][j] : dp[i][j - 1];
            dp[i][j] = grid[i][j] + max_prev;
        }
    }

    // Print the maximum monetary value at the bottom-right corner
    printf("%d\n", dp[n - 1][n - 1]);

```

```
    // Free allocated memory
    for (int i = 0; i < n; i++) {
        free(grid[i]);
        free(dp[i]);
    }
    free(grid);
    free(dp);

    return 0;
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:25 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:27 AM
Time taken	1 min 42 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a	g	**g**	**t**	**a**	**b**
s2	**g**	x	**t**	**x**	**a**	**y**	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

// Helper function to find the maximum of two numbers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int longestCommonSubsequence(char *s1, char *s2) {
    int m = strlen(s1);
    int n = strlen(s2);

    // Create a 2D DP table using Variable Length Arrays (VLA)
    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            // Base case: if either string is empty, LCS is 0
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            // If characters match, take diagonal value and add 1
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            // If characters don't match, take the max from top or left
            else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[m][n];
}

int main() {
    char s1[100], s2[100];

    // Read two input strings separated by space or newline
    if (scanf("%s %s", s1, s2) == 2) {
        printf("%d\n", longestCommonSubsequence(s1, s2));
    }

}
```

```
    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:28 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:29 AM
Time taken	1 min 36 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    // Read the size of the sequence only once
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // dp[i] stores the length of the longest non-decreasing subsequence ending at index i
    int dp[n];
    int max_len = 0;

    for (int i = 0; i < n; i++) {
        dp[i] = 1; // Every single element is a subsequence of length 1 by itself
        for (int j = 0; j < i; j++) {
            // Non-decreasing condition: arr[j] <= arr[i]
            if (arr[j] <= arr[i]) {
                if (dp[j] + 1 > dp[i]) {
                    dp[i] = dp[j] + 1;
                }
            }
        }
        // Track the overall maximum length found so far
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    printf("%d\n", max_len);

    return 0;
}
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:30 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:32 AM
Time taken	2 mins 24 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findDuplicate(int arr[], int n) {
    int low = 1, high = n - 1;

    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;

        // Count elements less than or equal to mid
        for (int i = 0; i < n; i++) {
            if (arr[i] <= mid) {
                count++;
            }
        }

        // If count exceeds mid, the duplicate is in the left half
        if (count > mid) {
            high = mid;
        } else {
            // Otherwise, it's in the right half
            low = mid + 1;
        }
    }

    return low;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int duplicate = findDuplicate(arr, n);
    printf("%d\n", duplicate);

    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔

	Input	Expected	Got	
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:33 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:34 AM
Time taken	50 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findDuplicate(int arr[], int n) {
    int low = 1, high = n;

    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;

        // Count how many elements are less than or equal to mid
        for (int i = 0; i < n; i++) {
            if (arr[i] <= mid) {
                count++;
            }
        }

        // If count exceeds mid, the duplicate lies in the left half
        if (count > mid) {
            high = mid;
        } else {
            // Otherwise, it lies in the right half
            low = mid + 1;
        }
    }

    return low;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int duplicate = findDuplicate(arr, n);
    printf("%d\n", duplicate);

    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔

	Input	Expected	Got	
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:34 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:35 AM
Time taken	1 min
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

void solveIntersection() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int arr1[n1];
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int arr2[n2];
    for (int i = 0; i < n2; i++) {
        scanf("%d", &arr2[i]);
    }

    int i = 0, j = 0;
    int first = 1; // Flag to manage space separation in output
```

```
    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) == 1) {
        while (t--) {
            solveIntersection();
        }
    }
    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:35 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:37 AM
Time taken	1 min 32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

void solveIntersection() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int arr1[n1];
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int arr2[n2];
    for (int i = 0; i < n2; i++) {
        scanf("%d", &arr2[i]);
    }

    int i = 0, j = 0;
    int first = 1; // Used to handle space-separated output formatting
```

```
    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) == 1) {
        while (t--) {
            solveIntersection();
        }
    }
    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:38 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:38 AM
Time taken	40 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int findPairWithDifference(int arr[], int n, int k) {
    if (n < 2) return 0;

    int i = 0, j = 1;

    while (i < n && j < n) {
        int diff = arr[j] - arr[i];

        // Check if the difference matches k and pointers are at different indices
        if (i != j && diff == k) {
            return 1;
        }
        // If difference is less than k, we need a larger value, so move j forward
        else if (diff < k) {
            j++;
        }
        // If difference is greater than k, we need a smaller value, so move i forward
        else {
            i++;
        }

        // Ensure i and j don't point to the same element
        if (i == j) {
            j++;
        }
    }

    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
```

```
40      }
41
42      int k;
43      scanf("%d", &k);
44
45      printf("%d\n", findPairWithDifference(arr, n, k));
46
47      return 0;
48  }
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Catherine Nivedita V 251001037-2025-IT

C2

Started on	Thursday, 17 September 2026, 10:38 AM
State	Finished
Completed on	Thursday, 17 September 2026, 10:41 AM
Time taken	2 mins 55 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    if (scanf("%d", &k) != 1) return 0;

    int i = 0, j = 1;
    int found = 0;

    // Two-pointer traversal
    while (i < n && j < n) {
        // Ensure i and j are distinct indices
        if (i == j) {
            j++;
            continue;
        }

        int diff = arr[j] - arr[i];

        if (diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++; // We need a larger difference, move the right pointer forward
        } else {
            i++; // We need a smaller difference, move the left pointer forward
        }
    }

    printf("%d\n", found);

```

```
40      return 0;
41  }
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course